TP application chapitre 2

Partie 1 : Index et `explain()`

Question 1 : Création de l'index sur `filiere`

```
// Création de l'index simple sur le champ filiere
db.etudiants.createIndex({ filiere: 1 })[cite: 13, 25]

// Affichage des index de la collection
db.etudiants.getIndexes()[cite: 13, 25]
```

Question 2 : Comparaison de la recherche avec `explain("executionStats")`

```
// Commande pour analyser la requête
db.etudiants.find({ filiere: "Informatique" }).explain("executionStats")[cite: 15, 25]
```

Question 3 : Index composé

```
// Création de l'index composé (filiere croissant, moyenne décroissante)
db.etudiants.createIndex({ filiere: 1, moyenne: -1 })[cite: 14, 25]

// Vérification avec la requête filtrée par filière et triée par moyenne décroissante
db.etudiants.find({ filiere: "Informatique" })
            .sort({ moyenne: -1 })
            .explain("executionStats")[cite: 14, 15, 25]
```

Partie 2 : Agrégation

Question 4 : Nombre d'étudiants et moyenne par filière

```
db.etudiants.aggregate([
  {
    $group: {
      _id: "$filiere",
      nbEtudiants: { $sum: 1 },
      moyenneFiliere: { $avg: "$moyenne" }
    }
  },
```

```
  {
    $sort: { moyenneFiliere: -1 }
  }
])[cite: 16, 18, 25]
```

Question 5 : Moyenne générale de toute la promotion

```
db.etudiants.aggregate([
  {
    $group: {
      _id: null, // Regroupe l'ensemble des documents en un seul résultat
      moyenneGenerale: { $avg: "$moyenne" }
    }
  },
  {
    $project: {
      _id: 0,
      moyenneGenerale: { $round: ["$moyenneGenerale", 2] }
    }
  }
])[cite: 22, 25]
```

Question 6 : Les 3 compétences les plus répandues en Informatique

```
db.etudiants.aggregate([
  { $match: { filiere: "Informatique" } },
  { $unwind: "$competences" },
  {
    $group: {
      _id: "$competences",
      total: { $sum: 1 }
    }
  },
  { $sort: { total: -1 } },
  { $limit: 3 }
])[cite: 18, 21, 25]
```

Question 7 : Nombre d'étudiants par ville ayant une moyenne ≥ 14

```
db.etudiants.aggregate([
  { $match: { moyenne: { $gte: 14 } } },
  {
    $group: {
      _id: "$adresse.ville",
```

```
      nbEtudiants: { $sum: 1 }
    }
  }
])[cite: 10, 18, 25]
```

Partie 3 : Validation JSON Schema

Question 8 : Définition de la règle de validation et test de conformité

1. Configuration de la validation

```
db.runCommand({
  collMod: "etudiants",
  validator: {
    $jsonSchema: {
      bsonType: "object",
      required: ["nom", "age", "competences"],
      properties: {
        nom: {
          bsonType: "string"
        },
        age: {
          bsonType: "int",
          minimum: 18,
          maximum: 30
        },
        competences: {
          bsonType: "array",
          minItems: 1,
          items: { bsonType: "string" }
        }
      }
    }
  }
})[cite: 23, 25]
```

2. Test avec un document invalide (Age hors limite et compétences vides)

```
db.etudiants.insertOne({
  nom: "Karim Hassan",
  age: 35, // Invalide : supérieur à 30
  competences: [] // Invalide : requiert au moins 1 élément (minItems: 1)
})[cite: 24, 25]
```

Résultat attendu :
MongoDB rejette l'insertion avec l'erreur :

> `MongoServerError: Document failed validation`